

Bienvenidos al examen en la convocatoria ordinaria de la asignatura de introducción a la programación II.

Recuerda que la sesión está siendo supervisada.

Una vez comenzado el examen en vuestro ordenador solo está permitido tener abiertas las siguientes aplicaciones:

- Editor de texto plano.
- Terminal para compilar/ejecutar código (Cygwin)
- Navegador web para acceder a blackboard.

En caso de tener otra aplicación abierta la calificación será de 0. Así no se podrá tener ninguna otra ventana de navegación que no sea la que contiene blackboard. Tampoco está permitido tener abiertas ninguna aplicación de correo, mensajería o escritorio remoto.

Dentro del editor de texto no se permitirán tener abiertas mas pestañas que una para cada una de los programas pedidos y los ficheros correspondientes

Dentro de la terminal solo estará permitido el uso de los siguientes comandos con cualquier opción: `pwd, cd, ls, gcc, gdb, ./<fichero_ejecutable.exe>`.

Os recordamos que no está permitido hablar ni hacer uso de dispositivos móviles así como de smart watches. Cualquier infracción de copia que se detecte supondrá la suspensión directa de la convocatoria independientemente de quien sea el programa o respuesta original.

En caso de cualquier aviso de la plataforma es necesario avisar a la persona correspondiente sobre ello.

FORMATO DE ENTREGA:

Antes de salir del examen, el alumno deberá de rellenar la respuesta a la pregunta, con el código correcto generado durante el examen. Para ello, se pueden seguir los siguientes pasos:

- Tomar el código en el editor de texto
- Se selecciona el código con el ratón y copiarlo
- Se pega en la respuesta en el cuadro habilitado para la entrega en el examen.

Además, se deberá subir el código c del programa creado. (El fichero .c)

Una vez hecho esto, se aprieta el botón "enviar" para acabar el examen.

CALIFICACIÓN:

Se valorará el uso de estructuras, enumerados, uniones, ficheros y una correcta gestión de la memoria. No es necesario que en los enunciados se escriba explícitamente que se deba usar memoria dinámica u algún tipo de estructura, el alumno debe saber en qué contexto debe usarse cada elemento.

En caso de necesitar un fichero en concreto, el alumno deberá crearlo en su máquina con los datos correspondientes.

En el caso de que se requiera usar alguna función NO está permitido modificar su cabecera.

Las salidas por consola de los ejercicios deben coincidir exactamente con los ejemplos mostrados en el enunciado. Esto incluye el formato, el orden y los textos indicados.

El alumno debe realizar los dos programas que a continuación se detallan. El primero tendrá una calificación de 4 sobre 10 y el segundo de 6 sobre 10.

FUNCIONES

Se pueden usar las siguientes funciones vistas en clase (no es obligatorio usar todo, pero está prohibido usar funciones no vistas en clase):

- `char *strcpy ( char *destino, const char *origen );`
- `char* strcat (char* destino, const char* fuente);`
- `int strcmp (const char* s1, const char* s2);`
- `int strcasecmp (const char* s1, const char* s2);`
- `void* memcpy(void* s1, const void* s2, size_t n);`
- `char *strstr (const char* s1, const char* s2);`
- `char *strtok (const char* s1, const char* s2);`
- `size_t strcspn (const char* s1, const char* s2);`
- `void *malloc(size_t size);`
- `void *realloc(void *ptr, size_t size);`
- `void free(void *ptr)`
- `int atoi (const char * str);`
- `int atof (const char * str);`
- `int fscanf(FILE *stream, const char *format, ...);`
- `getc(FILE *stream);`
- `int getchar(void);`
- `int fprintf(FILE *stream, const char *format, ...);`
- `FILE *fopen(const char *restrict pathname, const char *restrict mode);`
- `int fclose(FILE *stream);`

El límite de tiempo es 120 minutos.

Buena suerte

May the force be with you....

Pregunta 1 – 4 puntos

La empresa ParkControl desea desarrollar un sistema para gestionar la ocupación de un aparcamiento inteligente.

El aparcamiento estará formado por una serie de plazas.

Cada plaza puede encontrarse en uno de los siguientes estados:

- LIBRE
- OCUPADA
- RESERVADA

Las plazas ocupadas deberán almacenar la matrícula del vehículo correspondiente.

Al iniciar el programa se solicitará:

- número de filas del aparcamiento
- número de columnas del aparcamiento

Posteriormente se creará el aparcamiento.

El programa mostrará repetidamente el siguiente menú hasta que el usuario seleccione salir:

1. Aparcar vehiculo

2. Reservar plaza

3. Liberar plaza

4. Mostrar aparcamiento

0. Salir

Funcionalidad de las opciones del menú:

- Aparcar vehículo:

El usuario introducirá:

- fila

- columna
- matrícula

La plaza pasará a estado OCUPADA si estaba libre. En caso contrario se mostrará un mensaje de error.

- Reservar plaza:

El usuario introducirá:

- fila
- columna

La plaza pasará a estado RESERVADA si estaba libre. En caso contrario se mostrará un mensaje de error.

- Liberar plaza:

El usuario introducirá:

- fila
- columna

La plaza pasará a estado LIBRE.

- Mostrar aparcamiento:

Se mostrará el estado completo del aparcamiento utilizando:

- L para plazas libres
- O para plazas ocupadas
- R para plazas reservadas.

Ejemplo de aparcamiento:

L O R

L L O

R L L

Es obligatorio implementar las siguientes funciones:

- Lee dinámicamente una línea desde teclado

```
char* leerLineaDinamica ()
```

- Recibe el aparcamiento, una posición y una matrícula. Aparca un vehículo si la plaza está libre

```
int aparcarVehiculo(plaza_t** parking, int fila, int columna, char* matricula)
```

- Recibe el aparcamiento y una posición. Reserva una plaza si está libre

```
int reservarPlaza(plaza_t** parking, int fila, int columna)
```

- Recibe el aparcamiento y una posición. Libera una plaza del aparcamiento

```
int liberarPlaza(plaza_t** parking, int fila, int columna)
```

- Recibe el aparcamiento y lo muestra por consola

```
void mostrarParking(plaza_t** parking, int filas, int columnas)
```

Pregunta 2 – 6 puntos

La empresa SmartCity dispone de una red de sensores urbanos encargados de monitorizar distintos parámetros ambientales de la ciudad. Cada sensor genera eventos que son almacenados en un fichero de texto para su posterior análisis.

Cada línea del fichero contiene información sobre un evento generado por un sensor con el siguiente formato:

TIPO_SENSOR;VALOR;ESTADO

Donde:

- TIPO_SENSOR puede tomar uno de los siguientes valores:
 - TEMP
 - HUM
 - AIR
- VALOR es un número entero positivo correspondiente a la medición realizada por el sensor.
- ESTADO indica si la medición es válida:
 - OK
 - ERROR

Ejemplo de fichero de entrada:

TEMP;24;OK

HUM;80;OK

AIR;45;ERROR

TEMP;30;OK

HUM;70;OK

Realiza un programa que lea todos los eventos del fichero de entrada y los almacene en memoria. Posteriormente, el programa deberá generar un fichero de salida con un resumen de los sensores válidos (OK) indicando, para cada tipo de sensor, el número de muestras válidas y el valor máximo registrado.

Además, el programa deberá mostrar por pantalla todos los sensores cuyo estado sea ERROR.

El programa deberá ejecutarse con el siguiente formato:

`./analizaSensores.exe <fichero_entrada.txt> <fichero_salida.txt>`

Ejemplo de salida por pantalla:

Sensores con error:

AIR -> 45

Ejemplo de fichero de salida:

TEMP;2;30

HUM;2;80

AIR;0;0

Es obligatorio implementar las siguientes funciones:

- Lee dinámicamente una línea desde un fichero

```
char *leerLineaDinamicaFile(FILE *fd)
```

- Recibe una cadena de texto con el tipo de sensor y devuelve el valor correspondiente

```
tipoSensor_t obtenerTipoSensor(char *cadena)
```

- Recibe una cadena de texto con el estado del sensor y devuelve el valor correspondiente

`estadoSensor_t obtenerEstadoSensor(char *cadena)`

- Recibe una línea del fichero y devuelve un `eventoSensor_t` con los datos correspondientes al evento.

`eventoSensor_t parseaEvento(char *linea)`